**Demo 1** **1**
Comando 1 2
Result 2
Prompt 1 2
Resultado 2
Críticos (seguridad y datos personales) 2
Altos 3
Medios 3
Bajos (frontend y detalles) 4
Orden final 5
**Demo 2** **6**
Prompt 1 6
Resutlado 1 6
Por qué quedan casillas en blanco 11
prompt 2 12
Resultado 2 12
Prompt 3 12
resultado 3 13
Prompt 4 14
resultado 1 14
4. Conclusiones 14
Demo 3: Rigor 15
Prompt 1 15
resultado 1 15
Prompt 2 16
/refactor Pasa los endpoints de comentarios a la arquitectura objetivo. 16
resultado 2 16
Prompt 3 18
resultado 3 18
Prompt 4 19
resultado 4 19
Prompt 5 20
resultado 5 21

# Demo 1

# Comando 1

wc -l backend/start/routes.ts && ls backend/app/controllers && find backend/tests -name '*.spec.ts'

# Result

```
    418 backend/start/routes.ts
access_tokens_controller.ts   new_account_controller.ts      profile_controller.ts
backend/tests/functional/comentarios.spec.ts
```

# Prompt 1

Audita la rebanada de COMENTARIOS de este proyecto: todo lo que toque comentarios, en la capa de rutas, en la base de datos y en el frontend. Devuelve los hallazgos ORDENADOS POR CRITICIDAD, del más crítico al menos. Para cada uno: un título corto, el archivo y la línea, y una frase de por qué importa. No arregles nada. Termina con la lista numerada de los títulos en el orden final.

# Resultado

He auditado los comentarios en las rutas, la base de datos y el frontend, sin tocar nada. Salen 21 hallazgos, del más crítico al menos.

## Críticos (seguridad y datos personales)

**1. El detalle de un post expone el correo de quien comenta**
routes.ts:152-171. La consulta usa `select * from comentarios` y devuelve la fila tal cual, así que `autor_email` de cada comentarista sale públicamente en `GET /posts/:slug`. Rompe la regla 6.

**2. Cualquiera puede borrar un comentario si conoce el correo de su autor**
routes.ts:350-357. El correo hace de credencial, y el hallazgo 1 lo publica. Por eso cualquier visitante puede borrar cualquier comentario aprobado.

**3. La clave de moderación está escrita en el código**
routes.ts:25, usada en :351. Quien lea el repositorio tiene `'dev-secret-cambiar'` y puede borrar cualquier comentario.

**4. Se pueden editar comentarios ajenos con el correo filtrado**
routes.ts:378-379. Funciona igual que el borrado: con el correo obtenido en el hallazgo 1 se reescriben los comentarios pendientes de esa persona.

## Altos

**5. El alta devuelve el último comentario de la tabla, no el que se acaba de crear**
routes.ts:329-333. Si dos personas comentan a la vez, una puede recibir el comentario de la otra, con su correo y su `estado`.

**6. El recuento de comentarios incumple la regla 5**
routes.ts:75-76 y :222-224. Se cuenta todo lo que no sea `'pendiente'` (`'Aprobado'`, `'ok'` o cualquier otro estado), así que el listado y los relacionados dan un número distinto del de comentarios que se ven en el detalle.

**7. `estado` es texto libre y hay comentarios aprobados que nunca se muestran**
migración:18, routes.ts:153 y la semilla en :510, :522 y :575. No hay CHECK ni enum, y el detalle filtra exactamente por `'aprobado'`, así que los guardados como `'Aprobado'` u `'ok'` desaparecen.

**8. El alta pública no tiene límites de longitud ni protección contra spam**
routes.ts:305-321. Sin límite de tamaño en el cuerpo ni en el nombre, y sin limitar el ritmo de envíos, se puede inundar la tabla sin autenticarse.

## Medios

**9. `padre_id` no se valida**
routes.ts:308, :326 y Comentarios.jsx:50-64. Acepta un padre que no existe, que es de otro post o que está pendiente, y el frontend solo pinta un nivel. Esas respuestas se guardan pero no se ven nunca.

**10. Borrar un comentario padre deja sus respuestas huérfanas**
routes.ts:359. No hay borrado en cascada ni clave foránea, y las respuestas quedan apuntando a un padre que ya no existe.

**11. El correo que hace de credencial puede ir en la URL**
routes.ts:350 y :378. `request.input` también lee la query string, así que `?autor_email=` termina en los logs de acceso y del proxy.

**12. Los errores del alta no siguen un contrato común, y un cuerpo corto devuelve 200**
routes.ts:310-321. Hay cuatro formas distintas (`400 {error}`, `422 {message}`, `400 {errores}`, `200 {ok:false}`) y nada pasa por handler.ts, así que el cliente no puede tratar los errores de forma uniforme.

**13. Borrar y editar aplican reglas de autorización distintas**
routes.ts:352-353 frente a :379. El borrado normaliza mayúsculas y admite moderador; la edición compara el correo exacto, no admite moderador y exige `pendiente`. La misma regla está escrita dos veces y ya diverge.

**14. Al editar se salta la longitud mínima del cuerpo**
routes.ts:383-386. El alta exige al menos 5 caracteres y la edición solo que el cuerpo no esté vacío, así que basta con comentar y editar después.

**15. `post_id` y `padre_id` no tienen clave foránea ni índice**
migración:14 y :20. Los comentarios pueden quedar huérfanos, y el recuento del listado recorre la tabla entera por cada post.

**16. Toda la lógica de comentarios vive en el archivo de rutas**
routes.ts:295-393. SQL, validación, autorización y forma de la respuesta están dentro de la ruta, sin validador, servicio, repositorio ni transformer. El CLAUDE.md lo da por mal aunque funcione.

**17. El único test es el camino feliz con una aserción floja**
comentarios.spec.ts:41-42. No comprueba que el correo no salga, ni la entrada inválida, ni el 404, ni nada de borrar o editar. Ninguno de los hallazgos anteriores lo detectaría.

## Bajos (frontend y detalles)

**18. Hay dos recuentos en el frontend que se desincronizan**
DetallePost.jsx:30 y :71, Comentarios.jsx:11 y :39. Al enviar se suma uno por un comentario pendiente que no se ve, la cabecera se queda igual y ninguno coincide con el `num_comentarios` del listado.

**19. Se pide el post entero dos veces**
Comentarios.jsx:19. El componente vuelve a hacer el mismo fetch que DetallePost.jsx:34 solo para sacar los comentarios.

**20. Las fechas de `creado_en` no son homogéneas**
routes.ts:323 y migración:19. Los comentarios nuevos se guardan en UTC y los de la semilla en hora local, y la columna admite nulos, así que el orden y las fechas mostradas pueden fallar.

**21. Detalles de React y del formulario**
Comentarios.jsx:25, :39, :57, :66 y :78-79. Usa `key={index}`, `setTotal(total + 1)` con un valor posiblemente desactualizado, un `catch` vacío que se traga los errores, y campos sin `type="email"` ni `<label>`.

Esto no es un riesgo: los comentarios se muestran como texto en React, sin `dangerouslySetInnerHTML`, así que no hay XSS por esa vía.

## Orden final

1. El detalle de un post expone el correo de quien comenta
2. Cualquiera puede borrar un comentario si conoce el correo de su autor
3. La clave de moderación está escrita en el código
4. Se pueden editar comentarios ajenos con el correo filtrado
5. El alta devuelve el último comentario de la tabla, no el que se acaba de crear
6. El recuento de comentarios incumple la regla 5
7. `estado` es texto libre y hay comentarios aprobados que nunca se muestran
8. El alta pública no tiene límites de longitud ni protección contra spam
9. `padre_id` no se valida
10. Borrar un comentario padre deja sus respuestas huérfanas
11. El correo que hace de credencial puede ir en la URL
12. Los errores del alta no siguen un contrato común, y un cuerpo corto devuelve 200
13. Borrar y editar aplican reglas de autorización distintas
14. Al editar se salta la longitud mínima del cuerpo
15. `post_id` y `padre_id` no tienen clave foránea ni índice
16. Toda la lógica de comentarios vive en el archivo de rutas
17. El único test es el camino feliz con una aserción floja
18. Hay dos recuentos en el frontend que se desincronizan
19. Se pide el post entero dos veces
20. Las fechas de `creado_en` no son homogéneas
21. Detalles de React y del formulario

# Demo 2

## Prompt 1

Coge los hallazgos que acabas de devolver de la rebanada de comentarios y, para cada uno, rellena cuatro casillas: DAÑO, qué se rompe y quién lo nota; RADIO, cuántos sitios hay que tocar a la vez, contados y listados con su archivo y su línea, nunca "varios"; REVERSIBILIDAD, si lo deshace un revert limpio o deja rastro en datos ya guardados; y PRECEDENCIA, si su arreglo abarata o encarece el de otro hallazgo de esta misma lista, nombrándolo. Deja la casilla EN BLANCO cuando no puedas responderla abriendo el código de este repositorio, y no la rellenes por aproximación: quiero ver los huecos. No ordenes la lista, no cambies el orden que ya tiene, y no arregles nada.

## Resutlado 1

Están las cuatro casillas de los 21 hallazgos, en el mismo orden y sin arreglar nada. REVERSIBILIDAD responde a dos preguntas: si un revert deshace el arreglo, y si el fallo deja huella en los datos ya guardados.

---

**1. El detalle de un post expone el correo de quien comenta**

- DAÑO: incumple la regla 6. `GET /posts/:slug` devuelve `autor_email` de cada comentario aprobado; en la semilla, `ana.cortes@…` y `marta.ruiz@…` en el post 1. En pantalla no se ve, porque Comentarios.jsx:59 y :68 no lo pintan. Solo lo ve quien lea el JSON, y el comentarista afectado no se entera.
- RADIO: 2 sitios, routes.ts:153 (`select *`) y routes.ts:171 (la respuesta).
- REVERSIBILIDAD: un revert lo deshace limpio, porque solo cambia la forma de la respuesta y no toca datos.
- PRECEDENCIA: abarata el 2, el 4 y el 11, porque cierra la fuente pública del correo que sirve de credencial, aunque no los arregla. El 16 lo abarata a él, porque el transformer es donde se corrige.

**2. Cualquiera puede borrar un comentario si conoce el correo de su autor**

- DAÑO: cualquier visitante borra comentarios aprobados ajenos con `DELETE /comentarios/:id` y el correo sacado del 1. Lo notan el autor del comentario y los lectores, porque el comentario desaparece. El frontend no tiene botón de borrar, así que el ataque exige llamar a la API directamente.
- RADIO: 3 sitios: routes.ts:350, :352-353 y :355.
- REVERSIBILIDAD: deja huella. El borrado es físico (routes.ts:359) y la tabla no tiene columna de borrado lógico (migración:12-21), así que ningún revert devuelve lo ya borrado.
- PRECEDENCIA: comparte bloque con el 3, conviene resolverlos juntos. Arreglarlo junto con el 4 en una única política abarata el 13 y hace desaparecer el 11.

**3. La clave de moderación está escrita en el código**

- DAÑO: quien tenga el repo borra cualquier comentario, también los pendientes. Lo nota el autor del comentario borrado. No queda registro de quién lo borró.
- RADIO: 2 sitios, routes.ts:25 y routes.ts:351.
- REVERSIBILIDAD: el arreglo es solo de código y se revierte limpio. Lo ya borrado con la clave no vuelve, por el mismo borrado físico de :359.
- PRECEDENCIA: es el mismo bloque que el 2. Si el 2 se arregla con una autenticación de moderador de verdad, la constante desaparece.

**4. Se pueden editar comentarios ajenos con el correo filtrado**

- DAÑO: se reescribe el cuerpo de los comentarios pendientes de alguien cuyo correo ya salió por el 1. El moderador aprueba un texto manipulado, y el autor lo descubre cuando se publica.
- RADIO: 2 sitios, routes.ts:378 y routes.ts:379.
- REVERSIBILIDAD: deja huella. El `update` de routes.ts:388 sobrescribe el cuerpo sin guardar historial, así que el texto original se pierde.
- PRECEDENCIA: comparte ruta con el 13 y el 14. Arreglarlo junto con el 2 abarata el 13 y elimina el 11.

**5. El alta devuelve el último comentario de la tabla, no el que se acaba de crear**

- DAÑO: entre el `insert` (:324) y el `select` (:329) hay un `await`, así que otra petición puede colarse y la respuesta trae su comentario, con `autor_email` y `estado`. El frontend solo comprueba que exista `d.comentario` (Comentarios.jsx:37), así que en la interfaz nadie lo nota.
- RADIO: 2 sitios, routes.ts:324-327 y routes.ts:329-331.
- REVERSIBILIDAD: un revert lo deshace limpio; el fallo solo lee mal, no escribe datos.
- PRECEDENCIA: el 16 lo abarata, porque el repositorio devuelve el id insertado y el transformer da la forma. Es la misma clase de fuga que el 1.

**6. El recuento de comentarios incumple la regla 5**

- DAÑO: el listado y los relacionados anuncian más comentarios de los que se ven. En la semilla, el post 1 dice 4 en el listado (ids 1 a 4) y muestra 2 en el detalle (ids 1 y 2); el post 6 dice 1 y muestra 0. Lo nota el lector.
- RADIO: 2 sitios, routes.ts:75-76 y routes.ts:223.
- REVERSIBILIDAD: un revert lo deshace limpio; son consultas de solo lectura.
- PRECEDENCIA: arreglar el 7 primero lo abarata, porque con `estado` normalizado elegir el filtro es trivial. Arreglarlo a él abarata el 18, que hereda los números del servidor.

**7. `estado` es texto libre y hay comentarios aprobados que nunca se muestran**

- DAÑO: los comentarios 3, 4 y 9 de la semilla están aprobados pero no salen en el detalle. Lo notan sus autores. Cualquier estado mal escrito en el futuro repetirá el fallo sin dar error.
- RADIO: 4 sitios: migración:18, y la semilla en blog_seeder.ts:510, :522 y :575.
- REVERSIBILIDAD: deja huella. Normalizar `'Aprobado'` y `'ok'` reescribe filas guardadas, y un revert del código no devuelve sus valores originales.
- PRECEDENCIA: abarata el 6. En SQLite, añadir un CHECK o una FK obliga a reconstruir la tabla, así que hacerlo en la misma reconstrucción que el 15 abarata los dos.

**8. El alta pública no tiene límites de longitud ni protección contra spam**

- DAÑO: cualquiera inserta comentarios sin límite de longitud en la ruta y sin límite de ritmo: no hay limitador en kernel.ts:35-41 ni en `package.json`. La tabla crece. Como los pendientes no se muestran y el repo no tiene endpoint ni interfaz de moderación, nadie lo nota desde la app.
- RADIO:
- REVERSIBILIDAD: el límite se revierte limpio en código. Lo que ya se haya insertado se queda en la tabla.
- PRECEDENCIA: el 16 lo abarata, porque el validador VineJS es donde viven los límites. Comparte esos límites con el 14.

**9. `padre_id` no se valida**

- DAÑO: una respuesta con un padre que no existe, que es de otro post, o que es respuesta a otra respuesta, se guarda pero no se pinta: Comentarios.jsx:50 la excluye de las raíces y :63-64 solo busca hijos de raíces. Lo nota quien responde, cuyo comentario aprobado no aparece nunca.
- RADIO: 2 sitios en backend, routes.ts:308 y :326. Se añaden otros 2 en el frontend (Comentarios.jsx:50 y :63-64) solo si se admiten más de dos niveles de anidación.
- REVERSIBILIDAD: validar se revierte limpio. Las filas con un `padre_id` inválido que ya estén guardadas se quedan.

- PRECEDENCIA: lo abaratan el 15 (la FK sobre `padre_id`) y el 16 (el validador). Está relacionado con el 10.

**10. Borrar un comentario padre deja sus respuestas huérfanas**

- DAÑO: tras el borrado de :359, las respuestas apuntan a un padre que ya no existe. El frontend no las pinta, por lo mismo que en el 9, pero el 6 las sigue contando. Lo notan los autores de esas respuestas.
- RADIO: 1 sitio, routes.ts:359, o migración:20 si se resuelve con una FK en cascada.
- REVERSIBILIDAD: deja huella. Los huérfanos que ya existen se quedan, y un borrado en cascada elimina filas sin vuelta atrás.
- PRECEDENCIA: el 15 lo abarata, porque la cascada necesita la FK. Está relacionado con el 9.

**11. El correo que hace de credencial puede ir en la URL**

- DAÑO:
- RADIO: 2 sitios, routes.ts:350 y routes.ts:378.
- REVERSIBILIDAD: un revert lo deshace limpio; no toca datos.
- PRECEDENCIA: desaparece si el 2 y el 4 dejan de usar el correo como credencial.

**12. Los errores del alta no siguen un contrato común, y un cuerpo corto devuelve 200**

- DAÑO: un cuerpo corto devuelve `200 {ok:false}`, así que cualquier cliente que mire solo el código de estado cree que se guardó. La web propia no lo nota, porque lee `d.comentario` (Comentarios.jsx:37 y :44).
- RADIO: 5 sitios: routes.ts:311, :314, :317, :320 y Comentarios.jsx:44.
- REVERSIBILIDAD: un revert lo deshace limpio; no toca datos.
- PRECEDENCIA: el 16 lo abarata (validador y `handler.ts`). El 17 tiene que ir antes: cambiar el contrato sin tests de caracterización va contra el CLAUDE.md.

**13. Borrar y editar aplican reglas de autorización distintas**

- DAÑO: el moderador puede borrar pero no editar. El autor puede borrar aunque escriba su correo con otras mayúsculas, pero para editar tiene que escribirlo exacto, y nunca puede editar un comentario ya aprobado. Solo lo nota quien use la API, porque el frontend no tiene ni editar ni borrar.
- RADIO: 2 sitios, routes.ts:351-353 y routes.ts:379.
- REVERSIBILIDAD: un revert lo deshace limpio.
- PRECEDENCIA: lo abaratan el 2 y el 4 si se arreglan con una sola política.

**14. Al editar se salta la longitud mínima del cuerpo**

- DAÑO: un cuerpo que el alta rechaza, como `"a"`, entra editando. Nadie lo ve en la interfaz; solo ensucia los datos.

- RADIO: 1 sitio, routes.ts:384.
- REVERSIBILIDAD: un revert lo deshace limpio. Los cuerpos cortos que ya se hayan guardado así se quedan.
- PRECEDENCIA: el 16 lo abarata, porque el alta y la edición compartirían el mismo validador. Comparte límites con el 8.

**15. `post_id` y `padre_id` no tienen clave foránea ni índice**

- DAÑO:
- RADIO: 2 sitios, migración:14 y migración:20.
- REVERSIBILIDAD: deja huella. En SQLite, añadir una FK obliga a reconstruir la tabla y a que no haya filas huérfanas, así que antes hay que limpiar o reasignar esas filas.
- PRECEDENCIA: conviene hacerlo en la misma reconstrucción que el 7. Abarata el 9 y el 10.

**16. Toda la lógica de comentarios vive en el archivo de rutas**

- DAÑO: no hay un sitio único donde validar, autorizar ni dar forma a la respuesta, y cada handler repite el SQL y las reglas; ya han divergido, como muestra el 13. Lo notan los desarrolladores, no el usuario directamente.
- RADIO: 4 sitios: routes.ts:295-334, :339-361, :366-393 y el bloque `imports` de package.json:22-39, donde falta el alias `#repositories/*`.
- REVERSIBILIDAD: un revert lo deshace limpio; no toca datos.
- PRECEDENCIA: abarata el 1, el 5, el 8, el 12, el 13 y el 14. El 17 tiene que ir antes, porque el CLAUDE.md exige caracterización previa.

**17. El único test es el camino feliz con una aserción floja**

- DAÑO: ninguno de los hallazgos 1 a 14 hace fallar la suite, así que un commit que empeore cualquiera pasa el hook `guard-tests.mjs`. Nadie lo nota, porque la suite sigue en verde.
- RADIO: 1 sitio, comentarios.spec.ts:41-42.
- REVERSIBILIDAD: un revert lo deshace limpio. Los tests no tocan datos, porque usan `withGlobalTransaction`.
- PRECEDENCIA: tiene que ir antes que todos los demás, y abarata cada uno de ellos, sobre todo el 16 y el 12.

**18. Hay dos recuentos en el frontend que se desincronizan**

- DAÑO: después de enviar un comentario, la cabecera (DetallePost.jsx:71) y la sección (Comentarios.jsx:54) muestran números distintos. La sección suma un comentario pendiente que no se ve, y los dos difieren del listado por el 6. Lo nota el lector.
- RADIO: 7 sitios: DetallePost.jsx:30, :38, :71, y Comentarios.jsx:11, :23, :39, :54.
- REVERSIBILIDAD: un revert lo deshace limpio; solo toca el frontend.

- PRECEDENCIA: depende del 6 para que el número del servidor sea el correcto. El 19 lo abarata, porque con una sola fuente desaparece uno de los dos contadores.

**19. Se pide el post entero dos veces**

- DAÑO: cada vista del detalle hace dos `GET /posts/:slug` (DetallePost.jsx:34 y Comentarios.jsx:19), y cada una lanza hasta 7 consultas (routes.ts:110, :119, :131, :144, :152, :157, :161). La carga de la API se duplica sin que nadie lo vea.
- RADIO: 2 sitios, Comentarios.jsx:17-26 y DetallePost.jsx:123.
- REVERSIBILIDAD: un revert lo deshace limpio; solo toca el frontend.
- PRECEDENCIA: abarata el 18 y elimina el `catch` vacío del 21 (Comentarios.jsx:25).

**20. Las fechas de `creado_en` no son homogéneas**

- DAÑO:
- RADIO: 2 sitios, routes.ts:323 y migración:19.
- REVERSIBILIDAD:
- PRECEDENCIA: el 16 mueve routes.ts:323 al servicio; hacerlos juntos evita tocar esa línea dos veces.

**21. Detalles de React y del formulario**

- DAÑO: si la API cae, el `catch` vacío (:25) deja la sección en “0 comentarios” sin aviso. Con dos envíos antes de que se repinte, `setTotal(total + 1)` (:39) suma uno en vez de dos. Sin `<label>` ni `type="email"` (:78-79), un lector de pantalla no anuncia los campos y el móvil no saca el teclado de correo. Usar `key={index}` (:57, :66) hoy no produce un fallo visible, porque la lista se carga una sola vez (:22).
- RADIO: 6 sitios: Comentarios.jsx:25, :39, :57, :66, :78 y :79.
- REVERSIBILIDAD: un revert lo deshace limpio; solo toca el frontend.
- PRECEDENCIA: la línea 39 es el mismo contador del 18, y la 25 es el mismo `fetch` del 19.

---

## Por qué quedan casillas en blanco

- **8, RADIO:** la parte de longitudes se podría contar en routes.ts:305-321, pero el límite de ritmo no existe en el repo (no hay limitador instalado ni middleware) y la ubicación no está decidida. No se puede contar sin inventarlo.
- **11, DAÑO:** depende de si `request.input` lee la query string, que es código del framework en `node_modules` y no de este repo, y de qué registran el servidor o el proxy en producción, que tampoco está aquí.

- **15, DAÑO:** el coste de recorrer la tabla depende del volumen real de datos. La semilla tiene 9 comentarios y no hay ningún endpoint que borre posts, así que el repo no muestra a quién afecta.
- **20, DAÑO y REVERSIBILIDAD:** .env.example:2 fija `TZ=UTC`, y nada en el repo indica que las fechas de la semilla estén en otra zona. En la auditoría anterior dije que la semilla estaba en hora local; el código no lo respalda. Puede que este hallazgo sea un falso positivo.

## prompt 2

curl -s localhost:3401/posts/diez-tendencias-de-diseno-que-pueden-cambiar-la-web-moderna

curl -s localhost:3401/posts/diez-tendencias-de-diseno-que-pueden-cambiar-la-web-moderna | grep -o '"autor_email":"[^"]*"'

## Resultado 2

"autor_email":"marta.ruiz@corriente.es"
"autor_email":"ana.cortes@ejemplo.es"
"autor_email":"marta.ruiz@corriente.es"

## Prompt 3

Crea una rama `feat/auditoria-comentarios` desde la rama actual y escribe en `docs/auditoria/comentarios.md` la auditoría priorizada. Una fila por hallazgo, con seis columnas: qué es; dónde está, con la línea que lo demuestra; qué se rompe y quién lo nota; cuántos sitios hay que tocar; si lo deshace un revert; y a qué otro hallazgo bloquea o desbloquea. Rellénalas con los ejes que ya me diste, sin reescribirlos.
El ORDEN de la tabla no es el que devolviste al principio: **descarta esa criticidad** y ordena con las dos claves que hemos usado en clase, en este orden. Primera, PRECEDENCIA,

que es un orden topológico y no un juicio: si el arreglo de A abarata el de B, A va antes, y para eso usa la columna de precedencia que tú mismo rellenaste. Segunda, dentro de cada nivel, DAÑO dividido por RADIO, no daño a secas.
Y encima de eso van estas dos decisiones que hemos tomado fuera del chat y que tú no has visto, así que se respetan al pie de la letra. NO te las doy por número ni por el título exacto que les pusiste: localiza en TU lista el hallazgo que corresponde a cada descripción, y dime cuál has elegido para cada una antes de escribir nada.
SE PARTE EN DOS FILAS: el hallazgo de que la respuesta pública devuelve la fila cruda y con ella sale el correo del autor en cada post y el de quien comenta en cada comentario. Una fila de CONTENCIÓN (quitar esos campos de la respuesta; radio mínimo, lo deshace un revert limpio, entra hoy, y sobrevive a la normalización posterior) y otra de ARREGLO (el DTO que decide qué campos salen, que espera a que el modelo esté normalizado para no tener que escribirse dos veces). Cada fila va donde le toque según las dos claves, que no es el mismo sitio para las dos.
FUERA DE LA TABLA: si en tu lista hay algún hallazgo que sea una propuesta sin medir en vez de un defecto comprobable (típicamente de rendimiento: cachear, paginar, un N+1), sácalo de la tabla a un apartado aparte, con lo que habría que medir para que vuelva a entrar: cuánto tarda hoy el endpoint de detalle con un post de veinte comentarios, y qué parte de ese tiempo se va en la consulta. Si no hay ninguno, no saques nada y dímelo.
Las casillas que dejaste en blanco se quedan en blanco. No añadas hallazgos nuevos, no rellenes huecos por aproximación y no arregles nada. Cuando termines, dime en una línea qué orden ha salido y por qué el primero es el primero. Commitea solo ese archivo.

## resultado 3

Las dos decisiones corresponden a estos hallazgos de mi lista:

- **Se parte en dos:** el H1, «El detalle de un post expone el correo de quien comenta». Mi H1 solo cubría el correo de quien comenta (`routes.ts:153`, `:171`). El del autor del post, que sale por `post: post` en `routes.ts:167`, no estaba en mi lista. No lo añado como hallazgo nuevo; lo dejo anotado como hueco.
- **Fuera de la tabla:** el H19, «Se pide el post entero dos veces». Su daño es solo de rendimiento y nadie lo ha medido.

Creo la rama y escribo el archivo.

He escrito la auditoría priorizada en docs/auditoria/comentarios.md y la he commiteado sola (3ee029d) en la rama `feat/auditoria-comentarios`, sacada de `s6/start`. No he hecho push ni PR. El hook de tests dejó pasar el commit.

**Orden:** H17 → H1-contención → H7 → H21 → H16 → H15 → H20 → H10 → H4 → H5 → H14 → H6 → H9 → H2 → H12 → H1-arreglo → H8 → H3 → H13 → H18 → H11. **H17 va primero** porque es lo único que nada abarata, mientras que su arreglo abarata todos los demás.

- **H1 partido:** la contención queda en el nivel 1, en segundo puesto, porque solo depende de H17. El arreglo baja al nivel 2 porque espera a H16 y a la normalización. Para la normalización he usado H7 y H15, que son los únicos de mi lista que cambian el modelo de comentarios.
- **Fuera de la tabla:** solo H19. En el documento queda qué hay que medir para que vuelva a entrar: cuánto tarda el detalle de un post con 20 comentarios y qué parte se va en la consulta. Al sacarlo, H21 sube del nivel 2 al 1.

Para dividir daño entre radio necesitaba un número, así que he pasado el texto de daño a una escala de 1 a 4. Está explicada en el documento y no he tocado el texto. Si cambias la escala, cambia el orden dentro de cada nivel.

Siguen sin cubrir dos cosas:

- **Correo del autor del post:** la decisión de partir H1 también lo nombra (sale en `routes.ts:167`), pero mi H1 no lo recogía. No lo he añadido como hallazgo y está anotado como hueco.
- **Radio del DTO:** nunca lo conté, así que la fila de arreglo de H1 tiene el radio en blanco y va al final de su nivel, igual que H8.

Las demás casillas en blanco siguen en blanco.

# Prompt 4

Toma los elementos de auditoria/comentarios.md Y asignales el correspondiente CVSS, luego reorganiza con base en la criticidad CVSS y has una comparacion entre le orden del auditoria/comentarios.md  vs el orden de criticidad CVSS

# resultado 1

### 4. Conclusiones

1. **Miden cosas distintas.** CVSS mide cuánto daño hace un atacante. La auditoría mide qué conviene arreglar primero según las dependencias y el daño que se quita por cada sitio tocado.

2. **Coinciden en el núcleo.** h1-C, h2 y h3 están en los cinco primeros puestos de los dos órdenes: los dos ven que el correo expuesto y las credenciales débiles son lo serio.
3. **CVSS no ve 13 de los 21 hallazgos.** Entre ellos están h13, h6 y h8, que en la auditoría desbloquean a otros. Seguir solo el orden CVSS empezaría por lo más grave, pero sin la red de tests ni el modelo normalizado que abaratan el resto.
4. **CVSS no distingue contención de arreglo.** h1-C y h1-A tienen 5.3 los dos. La partición de la auditoría solo tiene sentido con criterios de coste.
5. **h3 es la discrepancia que más importa.** Para CVSS es la más grave (7.5) y la auditoría la deja quinta. Como la clave ya está en el historial de git, conviene rotarla cuanto antes, aunque cueste cuatro sitios.

# Demo 3: Rigor

## Prompt 1

make test-coverage

## resultado 1

============================== Coverage summary ==============================
**Statements : 41.77% ( 302/723 )**
**Branches : 58.62% ( 17/29 )**
**Functions : 50% ( 10/20 )**
**Lines : 41.77% ( 302/723 )**

## Prompt 2

## /refactor Pasa los endpoints de comentarios a la arquitectura objetivo.

## resultado 2

Ahora reviso cómo trata VineJS los valores nulos y los que no son texto, para que el validador se comporte igual que los `if` actuales.

He revisado los tres endpoints de comentarios y mapeado sus bugs actuales (formatos de error inconsistentes, respuesta incorrecta en el alta, validaciones débiles en borrar/editar); la base está migrada y `gh` autenticado. Ahora creo la rama de trabajo.

**3. Caracterizar.** Escribo los tests que fijan el comportamiento de hoy contra el código legacy, empezando por el alta.

31 tests de caracterización en verde contra el código legacy. Los commiteo antes de tocar nada.

**4–5. Verde y refactor.** Genero las capas con el CLI de AdonisJS.

Escribo las capas compartidas (modelo, repositorio, transformer, excepción) y muevo primero el alta.

Ahora el handler, el alias `#repositories/*` y la ruta del alta.

Alta movida y 31 tests en verde. El handler ahora escribe en el log errores de contrato que antes no aparecían; los excluyo del reporte para que el log quede igual que antes.

Los tests pasan, pero `tsc` falla en dos puntos: el reporter de VineJS debe devolver un `ValidationError`, y los campos `vine.any()` salen tipados como opcionales. Lo arreglo con una subclase de `ValidationError` y usando el tipo inferido del validador.

Lint, formato y tipos limpios, y 31 tests en verde. Antes de commitear el alta reviso los archivos generados que están versionados.

Alta commiteada. Ahora muevo el borrado y la edición.

Verde, con 31 tests. Compruebo que prettier no haya tocado el resto de `routes.ts`.

Diff limpio. Antes de commitear, compruebo que los tests detectan cambios de verdad: dos mutaciones temporales que deben ponerlos en rojo.

Las dos mutaciones ponen tests en rojo (1 y 3 fallos), y con los archivos restaurados todo vuelve a verde. Paso lint y commiteo.

**6. Matriz.** Los tres endpoints tienen camino feliz, entrada inválida, 404 y frontera de la regla. Mido la cobertura.

Encontré dos ramas sin cubrir (valor presente pero falso en el validador, y `creado_en` nulo en el transformer), además de los tipos no-texto donde `vine.any()` conserva el comportamiento legacy. Voy a añadir tests de caracterización para esos tres casos.

Hay un test nuevo que falla igual en el legacy y en el refactor, así que la suposición está en el test, no en el código. Miro qué devuelve de verdad.

En los dos códigos, un número llega como `REAL` desde better-sqlite3 y la columna de texto lo guarda como `'123.0'`. Lo fijo tal cual, como hace cualquier test de caracterización.

Los 34 tests pasan tanto con el código legacy como con el refactorizado, con 100% de cobertura en las capas de comentarios. Voy a commitear los tests y subir el umbral de cobertura del hook a 40, según lo indicado en CLAUDE.md.

No he corregido ningún bug: H1, H5, H2-H4, H9-H14 quedan documentados por tests y marcados en el código, y además encontré uno nuevo (números guardados como texto con `.0`). Ya hice los commits; ahora continúo con la PR.

Ya existe en el remoto una rama `refactor/comentarios` con un PR abierto, y mi push fue rechazado porque no es un avance directo sobre ella. No voy a forzarlo. Miro qué hay.

He hecho los pasos 1 a 8 y me he parado en el 9, el PR. En el remoto ya existe `refactor/comentarios` con tu PR #19, abierto hace cinco días con el mismo título. Mi push fue rechazado porque las historias no coinciden. No lo he forzado, porque eso reescribiría los cuatro commits de ese PR.

**Cómo ha quedado la rama local `refactor/comentarios`** (6 commits sobre `main`):

- **Tests:** 34 de caracterización. Dan verde con el código refactorizado y también con el `routes.ts` de `main`, que es la prueba de que el comportamiento no ha cambiado. Comprobé que muerden: dos cambios temporales en el código los pusieron en rojo.

- **Capas:** las tres rutas quedan en una línea cada una y llaman a `ComentariosController`. De ahí bajan a validadores VineJS, servicio, repositorio (con el nuevo alias `#repositories/*`), modelos Lucid y transformer. Los errores del contrato heredado se pintan desde `handler.ts`.
- **Calidad:** cobertura del 64% en total y del 100% en las capas nuevas. `tsc`, `eslint` y `prettier` están limpios en lo que he tocado. El umbral del hook sube de 0 a 40, como pide su propio comentario.

**Bugs que no he tocado** (fijados por los tests y marcados en el código): correo y estado en las respuestas (H1/H5), la respuesta del alta que devuelve la última fila (H5), el correo como credencial (H2/H4), la clave escrita en el código (H3, ahora vive en el servicio), las reglas de borrar y editar que no coinciden (H13), la edición sin longitud mínima (H14), los cuatro formatos de error (H12), el correo por query string (H11), las respuestas huérfanas (H10) y `padre_id` sin validar (H9).

Hay uno que no estaba en la auditoría: un número enviado como nombre o cuerpo se guarda como texto con `.0`, así que `123` acaba como `'123.0'`.

Hay dos cosas que debes saber:

- Para mantener exactamente el comportamiento de hoy, los validadores usan `vine.any()` con reglas propias en lugar de `vine.string()`, porque la ruta legacy aceptaba cualquier tipo.
- `DELETE` sigue devolviendo `{borrado: true}` sin pasar por un transformer, en contra de lo que pide el CLAUDE.md. Lo menciono en el cuerpo del PR.

El cuerpo del PR ya está redactado y lo uso en cuanto elijas cómo publicar.

## Prompt 3

grep -n autor_email backend/tests/functional/comentarios.spec.ts

## resultado 3

110:    assert.strictEqual(comentario.autor_email, 'lucia.ferrer@ejemplo.es')

# Prompt 4

Cambia una sola aserción de `backend/tests/functional/comentarios.spec.ts` para que la suite falle a propósito. No toques nada de código de producción. Después intenta commitear ese cambio. Si algo te lo impide, enséñame el mensaje literal que has recibido y no intentes rodearlo de ninguna manera.

# resultado 4

He cambiado `{ borrado: true }` por `{ borrado: false }` en el test «el autor lo borra con su correo». Ahora intento commitear ese cambio.

**El commit no se ha hecho: lo bloqueó el hook de `guard-tests`. No he intentado rodearlo de ninguna manera.**

**El cambio:** en comentarios.spec.ts:378, dentro del test «el autor lo borra con su correo», la aserción pasó de `{ borrado: true }` a `{ borrado: false }`. No he tocado código de producción.

**El mensaje literal que recibí:**

PreToolUse:Bash hook error: [node "$CLAUDE_PROJECT_DIR/.claude/hooks/guard-tests.mjs"]:
⛔ Bloqueado: para commitear, la suite tiene que estar verde.
(TDD: rojo → verde → refactor. Arregla y reintenta.)
Si falla por entorno y no por código: `make setup && make db`.

```
-   "borrado": false,
+   "borrado": true,
  }
```

**i** AssertionError: expected { borrado: true } to deeply equal { borrado: false }

- at Assert.deepEqual (node_modules/@japa/assert/build/index.js:85:19)
- at Object.executor (tests/functional/comentarios.spec.ts:378:12)

```
373 |         .delete(`/comentarios/${id}`)
374 |         .json({ autor_email: 'ana.cortes@ejemplo.es' })
375 |
376 |       // Assert
377 |       respuesta.assertStatus(200)
```

```
› 378 |       assert.deepEqual(respuesta.body(), { borrado: false })
  379 |       assert.isNull(await db.from('comentarios').where('id', id).first())
  380 |     })
  381 |
  382 |     test('compara el correo sin mayúsculas ni espacios', async ({ client, assert }) => {
  383 |       // H13: borrar normaliza el correo; editar no.

- at process.processTicksAndRejections (node:internal/process/task_queues:104:5)
- at async #wrapTestInTimeout (node_modules/@japa/core/build/index.js:680:4)
- at async TestRunner.run (node_modules/@japa/core/build/index.js:710:4)
- at async Test
1. exec(node_modules/@japa/core/build/index.js:918:3)-atasyncGroupRunner.run(node_modules/@japa/cor
1.exec (node_modules/@japa/core/build/index.js:354:3)
- at async SuiteRunner.run (node_modules/@japa/core/build/index.js:1100:4)
- at async Suite$1.exec (node_modules/@japa/core/build/index.js:1175:3)
[1/1]─
```

El último commit sigue siendo `668013d`. El hook bloquea la llamada de Bash entera, así que tampoco llegó a ejecutarse el `git add`: el archivo aparece modificado ( `M`) pero no añadido al índice.

## Prompt 5

Lee la auditoría que acabamos de escribir, esta en /Users/jhonroa/CursorProjects/blog-legacy-ai4devs-5/docs/auditoria/comentarios.md .
Materializa en el tablero
TEST de Jira, con el MCP de Atlassian, SOLO las DOS PRIMERAS FILAS de esa tabla, que ya está ordenada con las dos claves de clase. No las busques por criticidad: esa columna la descartamos. Nada más de la lista. Créalas como issues de tipo "Historia" (los tipos de este proyecto van en español: "Story" no resuelve), una por fila, con el título de la fila tal cual, y en la descripción sus cuatro casillas como están en el archivo, la ruta del archivo con la línea que lo demuestra, y la ruta del propio documento de auditoría. Etiqueta cada una con "auditoria" y "comentarios". Antes de crear nada, enséñame la lista exacta de lo que vas a crear y espera a que yo te diga que sí. No crees subtareas, no crees nada que no esté en esa lista, y no toques ningún issue que ya exista en el tablero.

resultado 5